
ONE-STEP LEARNING ALGORITHM SELECTION FOR CLASSIFICATION VIA CONVOLUTIONAL NEURAL NETWORKS

Sebastián Maldonado

Department of Management Control and Information Systems
School of Economics and Business
University of Chile, Santiago, Chile
sebastianm@fen.uchile.cl

Carla Vairetti

Universidad de los Andes
Chile
Facultad de Ingeniería y Ciencias Aplicadas cvairetti@uandes.cl

Ignacio Figueroa

Universidad de los Andes
Chile
Facultad de Ingeniería y Ciencias Aplicadas

August 26, 2025

ABSTRACT

As with any task, the process of building machine learning models can benefit from prior experience. Meta-learning for classifier selection leverages knowledge about the characteristics of different datasets and/or the past performance of machine learning techniques to inform better decisions in the current modeling process. Traditional meta-learning approaches first collect metadata that describe this prior experience and then use it as input for an algorithm selection model. In this paper, however, a one-step scheme is proposed in which convolutional neural networks are trained directly on tabular datasets for binary classification. The aim is to learn the underlying structure of the data without the need to explicitly identify meta-features. Experiments with simulated datasets show that the proposed approach achieves near-perfect performance in identifying both linear and nonlinear patterns, outperforming the conventional two-step method based on meta-features. The method is further applied to real-world datasets, providing recommendations on the most suitable classifiers based on the data's inherent structure

Keywords: Algorithm selection, Meta-learning, Machine Learning, Classifier selection¹.

1 Introduction

Meta-learning is a broad field of machine learning that originated decades before the rise of deep learning. Some previous studies in this field have learned from task properties, constructing *meta-features* that represent the datasets. The goal is then to transfer insights from the most similar tasks to a new task [6, 29]. The reasoning behind meta-learning has been extended to other relevant challenges associated with deep neural networks, such as transfer learning or

¹NOTICE: This is a preprint of a published work. Changes resulting from the publishing process, such as editing, corrections, structural formatting, and other quality control mechanisms may not be reflected in this version of the document. Please cite this work as follows: Maldonado, S., Vairetti, C., Figueroa, I. (2025) One-step learning algorithm selection for classification via convolutional neural networks. Information Sciences 721, 122610. DOI: <https://doi.org/10.1016/j.ins.2025.122610>.

multitask learning [19, 32]. However, these challenges are not related to this paper, which focuses on learning algorithm selection for classification [3, 14].

This paper proposes an approach for learning meta-features directly from tensor representations of data using convolutional neural networks (CNNs). Instead of using images as input, we use simulated tabular datasets with distinguishable linear and nonlinear patterns, framing it as a multiclass classification task. The main contributions are the following:

- A novel meta-learning approach is proposed that uses deep learning to extract meta-features from simulated tabular data, avoiding manual feature engineering. To the best of our knowledge, this is the first time such an approach has been proposed. In previous algorithm selection for classification approaches, meta-features were designed manually via feature engineering, and then used as inputs for a learning machine [3, 14].
- A pre-trained model capable of recommending suitable classifiers for tabular datasets without retraining or fine-tuning is introduced. Although deep learning has succeeded in tasks such as computer vision and text analytics, there are many applications where classification is usually performed on tabular datasets. Medicine and business analytics are examples of research fields that rely on traditional data to make predictions and decisions via machine learning (see, e.g., [7, 8]).
- The proposed method significantly reduces the time required for model comparison, making it practical for real-world applications in fields like medicine and business analytics. Suggesting suitable classifiers can alleviate the need for extensive model comparisons, which can be very time-consuming [3].
- This work opens a new research direction in algorithm selection for classification using representation learning, contributing to the broader field of meta-learning.

This study seeks to answer the following questions: (1) Can linear and nonlinear data structures be learned from raw tabular datasets using deep learning? and (2) If so, can this knowledge be leveraged to make suggestions on the most suitable machine learning method or methods for a specific task? The study is consistent with the reasoning behind the “No Free Lunch” (NFL) theorem in the sense that it may not be effective to gain knowledge from tasks to apply it to a completely unrelated dataset. However, existing meta-learning studies have shown the advantages of learning from prior experience [12]. This paper is organized as follows. Section 2 provides an overview of relevant meta-learning studies. The proposed meta-learning framework is presented in Section 3. Section 4 discusses the results obtained using simulated and benchmark datasets. Finally, Section 5 summarizes the primary conclusions and presents possible directions for future developments.

2 Prior Work on Meta-learning for Classifier Selection

Meta-learning is an umbrella term that encompasses several approaches that gain experience across tasks. The goal of meta-learning is to avoid time-consuming “trial and error” processes and make better choices when estimating machine learning models. Some examples of these approaches include the following (see [12] for a comprehensive literature review):

- *Learning from previous model estimations and evaluations:* Given a set of possible configurations Θ , the goal of this approach is to train a meta-learner based on previous model evaluations to make recommendations on a new task. In a hyperparameter search task, for example, the accuracy of a classifier can be estimated on different datasets (or variants of the same datasets using bootstrapping or cross-validation), and a meta-learner can be used to suggest an optimal hyperparameter configuration (or a set of candidate configurations, called a *portfolio* [12]). Note that a configuration can be a set of hyperparameters, network architectures, or pipeline components [12].
- *Transfer learning:* The reasoning behind this approach is to consider models that are trained with data from one or more sources as a starting point for developing a new model on a similar task. Although this idea has been applied to traditional machine learning methods, this approach has been particularly successful in deep learning (DL) [12]. In tasks such as object recognition and natural language processing (NLP), DL requires a large amount of data to achieve superior performance compared to other methods. A *pretrained model* can be constructed with publicly available data (e.g., Wikipedia or books in the case of NLP tasks [23] or ImageNet in the case of visual object recognition [21]), which is then adapted to the new task via *fine-tuning*.
- *Learning from meta-features:* The goal of this approach is to make better decisions during machine learning by learning from meta-features or properties that describe the datasets. These characterizations can be used for hyperparameter selection [22] or classifier selection [3, 14], among other tasks. A task similarity metric can be defined to transfer knowledge from one task to a new similar task, or a meta-learner can be constructed [2]. For example, in [3], a decision tree learner is trained on several meta-features that were constructed on more

than 100 datasets from the UCI Repository [4] and other sources. A multiclass classification problem with five classes related to five different classification algorithms is defined. Alternatively, information on dataset characteristics can be useful to infer the performance of feature selection methods [24].

Algorithm selection in machine learning can be seen as a targeted subtask of AutoML. AutoML refers to automated systems that aim to optimize the entire machine learning pipeline, including algorithm selection, hyperparameter tuning, feature engineering, and model evaluation. The goal is to provide an end-to-end solution that minimizes the need for expert intervention, often treating the problem as a black box [27]. In contrast, the present work focuses specifically on the algorithm selection problem. Instead of searching blindly, the aim is to learn patterns about which ML algorithms work best for which kinds of datasets, using meta-learning. It avoids the costly search process of full AutoML by leveraging prior knowledge. The method proposed in this study is related to the latter approach in the sense that learning is performed from characterizations of other tasks to make recommendations for suitable classifiers. However, instead of using a two-step approach by first constructing meta-features and then estimating a meta-learner, the proposed meta-learner is fed with tabular datasets directly. Different purposes can be distinguished in the creation of meta-features in the sense that specific measures aim to identify specific patterns, such as feature interdependence, class overlap, or task similarity:

- *Feature normality and dispersion:* Statistical measures used to describe a distribution can be considered to assess normality in the covariates. Some common examples are skewness $\frac{E(X-\mu)^3}{\sigma^3}$ and kurtosis ($\frac{E(X-\mu)^4}{\sigma^4}$), with μ and σ being the mean and standard deviation of variable X , respectively [30]. Other measures that can be related to feature normality are the interquartile range (IQR, the difference between quartile 3 and quartile 1) and the value of the 90th quantile [3]. Other statistics that can assess tendency and dispersion are the arithmetic mean, the geometric mean, the harmonic mean, the trimmed mean, the standard deviation, and the mean absolute deviation (MAD, $\frac{\sum |x_i - \mu|}{N}$, with $i = 1, \dots, N$ representing the samples) [3]. Finally, the Index of Dispersion (ID) indicates whether the data points are scattered or clustered [3].
- *Complexity:* Simple measures can be an indicator of the complexity of the learning task, such as the size of the dataset (number of rows and columns) and the number of classes. These measures can be related to the expected training times [20]. Alternatively, the percentage of missing values and outliers can be related to the complexity of the preprocessing step [3, 26].
- *Feature interdependence:* The level of redundancy in a dataset can be assessed using the Pearson correlation ($\rho = \frac{\sigma_{XY}}{\sigma_X \sigma_Y}$) or by analyzing the eigenvalues that result from applying the principal component analysis (PCA) method for feature extraction [20]. Some PCA-based meta-features are canonical correlations (square root of the eigenvalues), the first and last PC [3], and the skewness and kurtosis of the first PC [9].
- *Class overlap and feature relevance:* The meta-learning literature has reported different metrics to evaluate overall feature relevancy, such as the center of gravity (Euclidean norm between minority and majority classes) [3], the entropy of classes ($H(C) = -\sum_c \pi_c \log \pi_c$), the mean mutual information (MMI), the equivalent number of variables (ENV, ratio between the class entropy and the MMI), and the noise-signal ratio ($NSR = \frac{H(X) - M(C, X)}{M(C, X)}$) [3]. A large value for the latter metric suggests that the dataset contains several irrelevant/noisy variables, and therefore, its dimensionality can be reduced without affecting the classification accuracy [3].
- *Landmarks:* This strategy computes measures designed to assess task similarity, using the learners themselves to describe the dataset. The idea is to compare differences in terms of performance between configurations and/or datasets using simple classifiers, such as 1-NN or naive Bayes [10].

It is important to note that, although meta-learning has usually been applied for algorithm recommendation in classification [31, 36], other machine learning tasks in which meta-learning has shown good results include regression [18], time series analysis [16], data stream mining [25], and clustering [10]. The proposed meta-learning approach is discussed in the following section.

3 Proposed Classifier Selection Framework via CNNs

This paper proposes a novel meta-learning framework in which tabular datasets are introduced into a learning model directly, which differs from the traditional two-step approach that involves the construction of meta-features. In this sense, each tabular dataset is treated as an “image”, using CNNs for model training. The reasoning behind this approach is to avoid the loss of information that occurs when meta-features are constructed and subsequently used as inputs for a traditional classifier, such as decision trees [3]. The framework consists of four steps:

1. **Simulation of patterns:** Different training patterns are defined that can be related to the performance of learning models. These patterns can be simulated under different noise and class overlap conditions, and then used to feed a CNN in a supervised manner. For example, a logistic regression or a linear support vector machine (SVM) can be recommended when dealing with tabular datasets with a linear pattern. The most suitable classifier is a hyperplane, while kernel-based SVM, NNs, or random forests can be recommended when datasets at hand follow a nonlinear pattern. For a setting in which all tabular datasets are of similar size, each input consists of tuples in the form $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, where $\mathbf{x}_i \in \mathbb{R}^P$ and $y_i \in \{-1, 1\}$ for $i = 1, \dots, N$. In other words, the focus is limited to tabular datasets for binary classification tasks, but the approach can be extended easily to multiclass classification. For each simulated pattern $c = 1, \dots, C$, M different simulated datasets are considered. Therefore, the dimensions of the input tensor are $N \times (P + 1) \times (C \cdot M)$. This setting with homogeneous datasets is referred to as CNN1. Alternatively, a setting with datasets that are heterogeneous in the number of samples and variables can be simulated. This setting is more compatible with meta-learning because the goal is to transfer the knowledge gained from simulated data to real-world datasets, which are heterogeneous in size. Furthermore, such a setting allows the introduction of noise into the patterns in the form of irrelevant variables and/or samples that do not follow the simulated pattern, thus mitigating the risk of overfitting. Similar to CNN1, M different simulated datasets for each of the C patterns are considered. For each simulated dataset $mc = 1, \dots, MC$, N_{mc} and P_{mc} represent the sample size and number of covariates, respectively. This setting with heterogeneous datasets is referred to as CNN2. The first step is arguably the most challenging because only a comprehensive set of patterns under different conditions of noise would lead to an adequate application on real-world datasets. The success of the approach strongly relies on this step. As a first attempt, five different patterns that are well known in the machine-learning literature, such as the two moons or XOR patterns [34], are discussed. It is believed that this paper opens an exciting new line of research in which different sets of simulated patterns can be designed.
2. **Dealing with inputs of different sizes:** Because the proposed goal is to make recommendations for real-world tabular datasets, it is expected that datasets will all be of different sizes. Therefore, the training and test samples must be reshaped to a homogeneous size. This is only required for the CNN2 setting. Simple approaches are proposed for dealing with images of different sizes with CNNs, such as padding and principal component analysis (PCA). Padding is used to resize tabular datasets that are smaller than a target size by extending the area over which a CNN processes. This process is performed by introducing additional rows or columns of zeros [1]. PCA can, in contrast, reduce tabular datasets to a target size by finding (orthogonal) linear combinations of the original variables in such a way that the variance is maximized. Thus, the datasets can be shrunk while keeping their inherent structure [17]. Under the CNN2 setting, there are heterogeneous datasets $\{\mathbf{X}_{mc}, \mathbf{y}_{mc}\}_{mc=1}^{MC}$, with $\mathbf{X}$ and $\mathbf{y}$ representing the covariate matrix and target vector. In order to define a common dimensionality P , padding or PCA can be used. In the case of padding, the largest P_{mc} for all the simulated datasets ($P = \max_{mc}(P_{mc})$) can be selected and the missing dimensions filled with zeros for all the remaining datasets. In the case of PCA, a target dimensionality P can be defined, and P components are selected for each dataset. To define a common sample size N , padding or random undersampling can be used. In the case of padding, the largest N_{mc} for all the simulated datasets ($N = \max_{mc}(N_{mc})$) can be selected and the missing samples filled with zeros for all the remaining datasets. In the case of random undersampling, the minimum sample size can be defined ($N = \min_{mc}(N_{mc})$), discarding observations randomly in all the datasets until this number is reached. These two strategies can also be combined. After these preprocessing strategies, the input tensor for the next step has dimensions $N \times (P + 1) \times (C \cdot M)$.
3. **CNN training:** Often used for image-related tasks such as pattern recognition, segmentation, or object detection, CNNs are among the most popular deep learning variants [33]. CNNs are employed in this framework because they naturally handle tensor data and eliminate the need for separate feature extraction, which contributes to their superior performance in image recognition [13, 35]. Note that $\{\mathbf{X}_{mc}, \mathbf{y}_{mc}\}_{mc=1}^{MC}$ represent the inputs for the CNN model and are treated as images. The labels for these “images” are a vector of size MC indicating each of the C different patterns simulated in step 1. This vector is referred to as $\mathbf{z}$. It is important to make this distinction because the vectors $\mathbf{y}_{mc}$, which are the labels of the tabular datasets, can be confused with the labels of the tabular datasets. The classifier trained with simulated data is referred to as $\mathcal{C}$.
4. **Application to real-world datasets:** The final step consists of applying the CNN model $\mathcal{C}$ to real-world datasets to identify one of the simulated patterns that were considered during training. The pattern found can be linked to one or more suitable classifiers for this pattern. It is important to assess the confidence of the classifier in its choice; therefore, the probabilistic output of the network must be analyzed using a softmax function. For example, a learning task with five simulated patterns is considered in our experimental design. If the largest predicted probability for a given real-world dataset is 0.3, then the model is undecided, and no recommendation can be made because this probability is too close to a random choice (0.2 for a 5-class task). In contrast, the model is certain when the probability of a given class is near 1, leading to a trustworthy

recommendation. Note that step 2 is required to preprocess the real-world datasets. This preprocessing should be consistent with the one performed for the training datasets; i.e., the real-world datasets should also have input sizes N and P . Assuming that there are T real-world datasets, each (heterogeneous) input is referred to as $\{\mathbf{X}_t^r, \mathbf{y}_t^r\}_{t=1}^T$. After preprocessing, the dimensionality of the tensor for the application of the model is $N \times (P + 1) \times T$. Notice that assigning patterns $\mathbf{z}^t$ to the real-world datasets is not the ultimate goal of the proposed framework; the goal is to recommend one or more classifiers. To this end, the performance of various classifiers for tabular datasets with the simulated data is evaluated and the best ones are identified. Once a pattern is assigned to a real-world dataset, the classifier that performs best for this pattern is recommended. For example, if a random forest performs best for pattern $C = 2$ and the classifier $\mathcal{C}$ assigns this pattern to the real-world dataset $t = 1$ ($z_1^t = 2$), then the random forest is recommended as the binary classifier for this particular task. In contrast to the first stage of the framework, which follows a standard supervised scheme, a tailored strategy needs to be defined to evaluate the success of the transfer learning approach. For this stage, the aim is to assign labels to real-world tabular datasets that do not necessarily follow a clear linear/nonlinear pattern. The assumption is that these labels can be linked to classifiers that can perform well for these patterns.

4 Experimental Results

To validate the proposed meta-learning framework, CNN models were first trained with simulated data exhibiting different linear and nonlinear patterns and their performances were compared with the traditional approach based on meta-features proposed in [3]. The results of these experiments are shown in Section 4.1. The application of CNNs with real-world tabular datasets is reported in Section 4.2. Finally, Section 4.3 provides a discussion that summarizes the main experimental findings.

4.1 Construction of the Classifier Selection Model

A total of 50,000 tabular datasets were simulated with five different patterns ($C = 5$, $M = 10,000$ datasets each). Five two-dimensional patterns that act as classes for the meta-learner were defined: a linear pattern (C1), an XOR pattern (C2), a two-moons pattern (C3), a “sandwich” pattern (C4), and a quadratic pattern (C5). The following two-dimensional patterns were defined:

- **Class 1 - linear pattern:** Two Gaussian functions are combined, one for each class, varying the mean and standard deviation of each Gaussian to create different overlap and noise conditions.
- **Class 2 - XOR pattern:** Four Gaussian functions are combined, two for each class, varying the mean and standard deviation of each Gaussian to create different overlap and noise conditions. The four Gaussians shape the well-known XOR pattern related to the exclusive disjunction logical operation. This nonlinear pattern is also known as “checkerboard”.
- **Class 3 - “Two moons” pattern:** This well-known synthetic dataset constructs a swirl pattern shaped like two moons. The conditions are varied in terms of noise and overlap.
- **Class 4 - “Sandwich” pattern:** Samples are simulated using one Gaussian function interspersed with two other Gaussians related to a second class. This results in a nonlinear pattern in the form of a “sandwich”. The mean and standard deviation of each Gaussian distribution are varied to create different overlap and noise conditions.
- **Class 5 - Quadratic pattern:** Samples are simulated based on two quadratic functions, one for each class, with a marginal overlap. The conditions are varied in terms of noise and overlap.

Figure 1 shows the five simulated patterns.

Two different sets of experiments are defined based on the five patterns for model training. First, two-dimensional matrices of similar size (i.e., $N = 1000$ and $P = 2$) are considered. This meta-learning approach is referred to as CNN1. Heterogeneous datasets in terms of the number of samples and variables are also considered: the patterns were generated using a random number of samples between 800 and 1400 and 0 to 4 irrelevant variables (Gaussian noise) were introduced, adding them to the two-dimensional patterns (i.e., $m = [800, 1400]$ and $p = [2, 6]$). This approach is referred to as CNN2. The supervised learning task involves predicting the correct pattern of the datasets (i.e., a multiclass problem with $C = 5$), and the inputs are two-class datasets, where the two classes are perfectly balanced

To highlight the performance differences of the various patterns, several well-known classifiers were used, evaluating performance on a test set of simulated tabular datasets. The following classification methods and their respective hyperparameter values were considered:

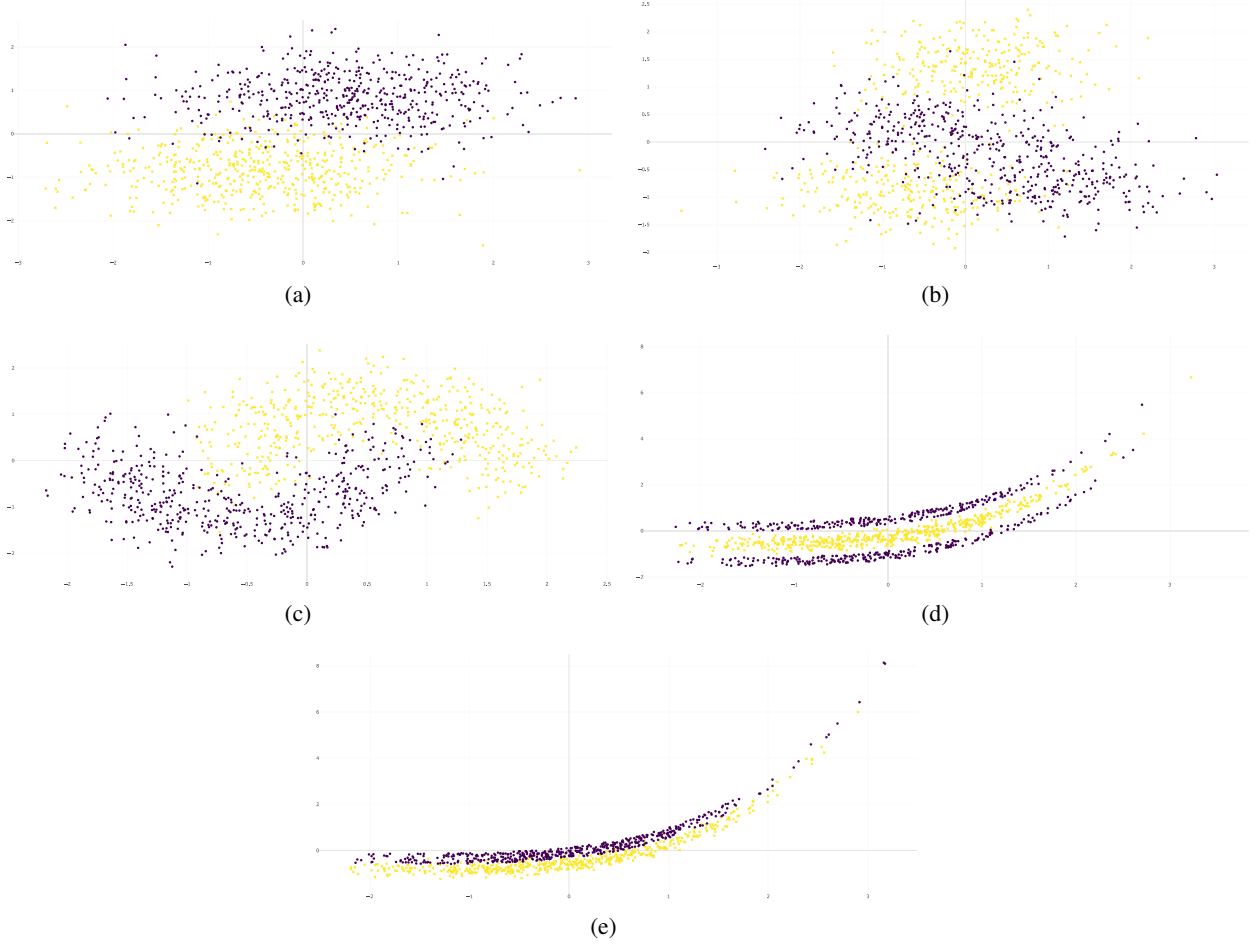


Figure 1: Simulated patterns for the meta-learning framework.

- *Logistic regression (Logit)*: This statistical method is suitable for a linear pattern because it constructs a separating hyperplane. The slopes of this function are given by a vector of coefficients β .
- *Decision tree (DT)*: This method performs a branching process in a hierarchical manner until a pruning criterion is met. The Gini index was considered for splitting, while three different values were considered for the complexity parameter (default pruning parameter values of the ‘rpart’ R implementation) $cp = \{0, 0.03, 0.81\}$.
- *k-nearest neighbor (k-NN)*: This classifier predicts new samples according to the labels of a neighborhood of size k that were obtained from the training set, with $k = \{5, 7, 9\}$.
- *Random forest (RF)*: This method constructs an ensemble of decision trees via bagging. The same criteria for the DT classifier were considered, constructing 500 trees with two randomly selected variables.
- *Artificial neural network (ANN)*: A shallow ANN classifier with one hidden layer was constructed. 1, 3, and 5 hidden units were explored and the following values for the weight decay parameter: $\epsilon = \{0, 0.0001, 0.1\}$.
- *Support vector machine (SVM)*: The standard kernel-based SVM model with Tikhonov regularization and hinge loss was considered. The following values for the regularization parameter were explored: $C = \{0.25, 0, 5, 1, 2, 4\}$, while the kernel width σ was held constant at a value of 1.004454.

Table 1 shows the area under the curve (AUC) measure in the test set for all the methods and simulated patterns (C1 to C5). The largest AUC value is highlighted in bold type, while the second largest AUC appears underlined. Because all the patterns are binary classification problems, AUC is a suitable performance metric that does not rely on a threshold and can balance type-I and type-II errors adequately [11].

Table 1 shows that the top classification approaches vary depending on the simulated pattern:

Table 1: Predictive performance for the five simulated patterns.

Classifier	C1	C2	C3	C4	C5
Logit	0.938	0.590	0.954	0.609	0.904
DT	0.859	0.803	0.890	0.845	0.903
k -NN	0.909	0.885	0.965	<u>0.966</u>	0.970
RF	0.910	<u>0.884</u>	<u>0.966</u>	<u>0.962</u>	<u>0.981</u>
ANN	<u>0.926</u>	<u>0.577</u>	<u>0.953</u>	0.691	<u>0.867</u>
SVM	0.920	0.882	0.968	0.981	0.985

- C1: Logit is the best classifier, followed by ANN.
- C2: k -NN is the best classifier, followed by RF.
- C3: SVM is the best classifier, followed by RF.
- C4: SVM is the best classifier, followed by k -NN.
- C5: SVM is the best classifier, followed by RF.

At this stage, the proposed approach can match the expected classifiers in the sense that the logistic regression performs best for the sole linear pattern (C1), while nonlinear methods (kernel-based SVM, random forest, and k -NN) perform best with the remaining four nonlinear patterns. For these patterns, the logit classifier does not perform well. Although it is clear that “there is no free lunch”, recommendations for classifiers can still be made if the patterns are identified correctly. Note that the goal of training CNNs with a heterogeneous dataset in terms of size is to consider the fact that the goal is to apply the final model to real-world data, which are inherently heterogeneous. Training with tabular datasets of different sizes (CNN2) is then a more realistic approach to meta-learning. However, transformations are required before model training (step 2 of the proposed framework). As mentioned in the previous section, two simple strategies were combined to manage “images” of different sizes (simulated tabular datasets in the proposed case): (1) PCA to shrink the datasets that are larger than a target average sample size (1099 observations for each tabular dataset, where a total of 1099 components are selected); and (2) padding to create additional space within a dataset smaller than the target average size. Padding is also used to achieve the target size of 7, which is the maximum size of a tabular input $\mathbf{X}$, $\mathbf{y}$ with the two-dimensional pattern, four irrelevant variables, and the label vector $\mathbf{y}$ included. The network used is a straightforward two-stage architecture. The first stage is used for feature extraction and learning and consists of two Conv2D layers with 32 filters and a kernel size of 5x5, followed by a dropout layer with a rate of 0.25 and then two Conv2D layers with 64 filters each and a kernel size of 3x3. The first two Conv2D layers use a kernel size of 5x5 for more general feature extraction, while the third and fourth use 3x3 kernels to extract more detailed features from the data. The second stage performs the classification task and begins with a flattened layer, followed by a dense layer with 256 neurons, a dropout layer with a rate of 0.5, another dense layer with five neurons, and finally a softmax layer with five outputs for the five simulated patterns proposed in this experimental setting. The network was trained with a batch size of 1000 tabular datasets and a maximum of 10 epochs, allowing early stopping. A standard validation approach is considered, in which the validation set is used to monitor the predictive performance of the model (validation loss). Adam was used as the optimizer, which has shown excellent empirical results in terms of efficiency, scalability, and performance (i.e., faster convergence to a minimum) [1]. Adam is a stochastic gradient descent method with adaptive learning rate, allowing a more effective and smooth learning process. The following values were considered for the optimization process with Adam: $\alpha = 1e - 03$ (initial learning rate), $\beta_1 = 0.9$ and $\beta_2 = 0.999$ (parameters that control the exponential decay rates of the moving averages), and $\epsilon = 1e - 07$ (conditioning parameter). These parameter values were selected according to the original Adam paper [15]. The proposed methodology was implemented in the TensorFlow Python library. Table 2 shows the performance of five different meta-learning strategies by considering various multiclass metrics: micro and macro averages for the precision, recall, and f1 (harmonic mean of the precision and recall). The proposed four CNN variants and the meta-learning approach suggested in [3] (MF+DT) are considered, which consists of constructing several meta-features and using a decision tree for model training with the meta-features as inputs. This latter strategy includes several meta-learning studies given the set of meta-features considered in this study (see [3, 7]). The heterogeneous dataset used to train CNN2 is considered to construct the meta-features.

Table 2 shows that the two proposed CNN variants achieved similar performances with nearly perfect classification. It can be concluded that the proposed approach can learn from various simulated patterns and identify them in other unseen tabular datasets with marginal variations. In contrast, the two-step approach that constructs meta-features and then implements a classifier is clearly inferior in terms of its predictive capabilities, with an accuracy of approximately 75% on average. While Table 2 demonstrates the CNN models’ ability to identify known patterns in unseen simulated data, it should be interpreted with caution, as both training and validation sets share the same underlying patterns. The

Table 2: Predictive performance for the various meta-learning approaches.

Perf. Measure	CNN1	CNN2	MF+DT
precision (macro)	0.996	0.986	0.790
recall (macro)	0.996	0.985	0.752
f1-score (macro)	0.996	0.985	0.745
f1-score (micro)	0.996	0.985	0.752

true test of generalization lies in the results in Section 4.2, where the model successfully recommends classifiers for real-world datasets it never encountered during training.

4.2 Application on benchmark datasets

Next, the ability of the proposed model to make suggestions for suitable classifiers is explored. The proposed CNN2 approach is compared with the alternative strategy based on meta-features (MF+DT) on well-known binary classification datasets from the UCI Repository [4]. CNN2 is chosen because its construction is designed to manage heterogeneous datasets, although CNN1 achieved marginally better classification results. Also, CNN2 and MF+DT consider the same input information and are therefore comparable. Table 3 shows relevant metadata for all the benchmark datasets, including the number of examples m , the number of variables n , and the percentage of examples in each class (min.,maj.), and the imbalance ratio (IR), computed as the fraction of the majority class and minority class samples.

Table 3: Descriptive statistics for all the benchmark datasets.

ID	dataset	m	n	%class(min,maj)	IR
ds1	abalone7	4177	8	(9.3, 90.7)	9.7
ds2	australian-credit	690	14	(44.5,55.5)	1.2
ds3	banknote-auth	1372	4	(44.5,55.5)	1.2
ds4	breast-cancer	569	30	(37.3,62.7)	1.7
ds5	bupa-liver	345	6	(42, 58)	1.4
ds6	german-credit	1000	24	(30.0,70.0)	2.3
ds7	hearth-statlog	270	13	(44.4,55.6)	1.3
ds8	horse-colic	300	27	(33.0,67.0)	2.0
ds9	image-1	2310	19	(14.3, 85.7)	6
ds10	image-5	2310	19	(38.1,61.9)	1.6
ds11	ionosphere	351	34	(35.9,64.1)	1.8
ds12	monk-2	432	6	(47.2,52.8)	1.1
ds13	oil-spill	937	49	(4.4,95.6)	21.9
ds14	phoneme	5404	19	(29.3,70.7)	2.4
ds15	pima-diabetes	768	8	(34.9,65.1)	1.9
ds16	ring	7400	20	(49.5,50.5)	1.0
ds17	solar-flares-M	1389	10	(4.9,95.1)	19.4
ds18	sonar	208	60	(46.6,53.4)	1.4
ds19	splice	1000	60	(48.3,51.7)	1.1
ds20	titanic	2201	3	(32.3,67.7)	2.1
ds21	waveform	5000	21	(33.1,66.9)	2.0
ds22	yeast02579vs368	1004	8	(9.9, 90.1)	9.1
ds23	yeast5	1484	8	(3.0, 97.0)	32.78

The tabular datasets exhibit important differences in terms of size and imbalance ratio, which is important to assess meta-learning approaches properly (see Table 3). Note that these datasets have no “class” in the sense that they have no known linear/nonlinear pattern. Therefore, multiclass classification performance cannot be computed. However, the binary classification performance for each tabular dataset can first be computed and then compared with the patterns suggested by both CNN and MF+DT. The results of the various classifiers on the 23 datasets are shown in Table 4. The best classifier for each dataset is highlighted in bold type.

Table 4 shows that no method outperforms others in terms of performance. In contrast to the performance of these classifiers with simulated data (see Table 1), random forest and k -NN achieved a better performance than SVM. These

Table 4: Predictive performance for the various benchmark datasets.

ID	Logit	DT	k -NN	RF	ANN	SVM
ds1	0.777	0.82	0.892	0.864	0.793	0.822
ds2	0.865	0.863	0.844	0.865	0.858	0.866
ds3	0.982	0.977	0.997	0.986	1	1
ds4	0.544	0.644	0.56	0.613	0.604	0.5
ds5	0.51	0.594	0.719	0.671	0.676	0.702
ds6	0.647	0.514	0.616	0.666	0.527	0.617
ds7	0.701	0.755	0.594	0.79	0.759	0.775
ds8	0.5	0.519	0.625	0.745	0.5	0.5
ds9	0.919	0.96	0.994	0.987	0.978	0.981
ds10	0.504	0.556	0.65	0.611	0.559	0.536
ds11	0.761	0.786	0.886	0.935	0.912	0.932
ds12	0.771	0.822	0.93	0.941	0.906	0.899
ds13	0.5	0.5	0.577	0.5	0.5	0.5
ds14	0.667	0.753	0.842	0.804	0.745	0.792
ds15	0.538	0.498	0.591	0.634	0.599	0.585
ds16	0.824	0.82	0.813	0.821	0.821	0.821
ds17	0.769	0.817	0.952	0.929	0.793	0.826
ds18	0.68	0.681	0.85	0.771	0.757	0.68
ds19	0.743	0.723	0.787	0.783	0.827	0.807
ds20	0.692	0.675	0.697	0.677	0.695	0.695
ds21	0.867	0.803	0.84	0.862	0.866	0.867
ds22	0.876	0.93	0.941	0.946	0.865	0.913
ds23	0.957	0.976	0.979	0.981	0.975	0.972

results confirm the advantages of RF when facing noisy mixed-type data (numerical and dummy variables). In contrast to the first stage of the framework, which follows a standard supervised scheme, a tailored strategy needs to be defined to evaluate the success of the transfer learning approach. For this stage, the aim is to assign labels to real-world tabular datasets that do not necessarily follow a clear linear/nonlinear pattern. The assumption is that these labels can be linked to classifiers that can perform well on these patterns. The evaluation approach is again supervised: a classifier recommendation can be successful or not. Success is defined as follows: For a given pattern/class C1 to C5, the two best classifiers out of a total of six are identified. For a real-world dataset, the pattern is inferred with the CNN model, thus recommending these two classifiers as the best choices for it. Next, the six classifiers are applied to the tabular dataset and the performance is evaluated, identifying the best classifier (bc). A “hit” occurs when one of the two classifiers recommended by the predicted class coincides with bc. Otherwise, the recommendation was not successful. Notice that this validation scheme differs from unsupervised approaches based on cluster purity (see, e.g., [5]). Next, the results of the proposed CNN approach for meta-learning are reported in Table 5. For each dataset, the predicted probability of belonging to a given training pattern is shown. The largest predicted probability for a given dataset is highlighted in bold type. The best classifier (bc) found in Table 4 is also reported, and whether the recommendation is successful or not (column “hit”). For ds1, for example, C2 is the predicted class with a probability of 0.978, and therefore, the recommendations for this dataset based on the simulated data are first k -NN and then RF (see Table 1). The best-performing classifier is k -NN; therefore, the recommendation is valid. A hit is highlighted with the symbol ✓, also including the relative position of the recommended classifier (1 or 2). In contrast, for dataset ds4, the recommendations are also k -NN and RF (class C2, with a confidence of 0.947), but the best classifier is DT, and therefore, the recommendation is not successful (hit=✗). Note that there is a tie for bc in three of the 23 datasets. In such cases, the best classifier that is considered a hit appears underlined in case one of the two best classifiers is indeed recommended by the CNN.

Table 5 shows the advantages of the CNN: recommendations are successful 78.2% of the time (18 of the 23 datasets). Because a random recommender would achieve a 33.3% success rate at suggesting two of the six classifiers, the proposed meta-learning approach clearly outperforms a random model. Of the 18 hits, 13 correspond to the second-best model, while the remaining five correspond to the best model. However, at least two classifiers should be recommended because the performance of the top method is typically similar to that of the second-best method. Table 5 also shows the confidence of the CNN in predicting the training pattern: the largest probability is below 0.8 only in three of the 23 datasets. This result demonstrates that the method indeed identifies one of the various patterns in the datasets. Finally, the performance of the proposed method is compared with the traditional two-step approach. Table 6 shows the results

Table 5: Predictions for the proposed CNN approach.

ID	C1	C2	C3	C4	C5	bc	hit
ds1	0.022	0.978	0	0	0	k-NN	✓(1)
ds2	0.965	0.026	0	0	0.009	SVM	✗
ds3	0.044	0	0	0.057	0.898	<u>ANN/SVM</u>	✓(1)
ds4	0.041	0.947	0.001	0.001	0.01	DT	✗
ds5	0.163	0.679	0.02	0.026	0.111	k-NN	✓(1)
ds6	0.028	0.007	0.008	0.142	0.815	RF	✓(2)
ds7	0.09	0.869	0.009	0.008	0.025	RF	✓(2)
ds8	0	0	0.006	0.078	0.916	RF	✓(2)
ds9	0.002	0.15	0	0.848	0	k-NN	✓(2)
ds10	0	0.004	0	0.996	0	k-NN	✓(2)
ds11	0.007	0.992	0	0	0.001	RF	✓(2)
ds12	0.002	0.998	0	0	0	RF	✓(2)
ds13	0	0	0	0.994	0.006	k-NN	✓(2)
ds14	0	1	0	0	0	k-NN	✓(1)
ds15	0	0	0	0	1	RF	✓(2)
ds16	0	0.98	0.02	0	0	logit	✗
ds17	0.028	0.444	0	0.528	0	k-NN	✓(2)
ds18	0.126	0.559	0.075	0.143	0.097	RF	✓(2)
ds19	0.038	0.003	0.004	0.016	0.939	ANN	✗
ds20	0.008	0.147	0	0.845	0	k-NN	✓(2)
ds21	0	0.069	0.931	0	0	<u>logit/SVM</u>	✓(1)
ds22	0.019	0.01	0.895	0.064	0.012	RF	✓(2)
ds23	0.99	0.008	0	0.002	0	RF	✗

when a DT is applied to meta-features. The identified training pattern is highlighted with the symbol ✗. In the case of ties, the best classifier (bc) that is recommended by the meta-learning method appears underlined. Note that landmarks are not included because they consider information that is not available for the proposed approach (e.g., the performance of the various traditional classifiers).

Table

6 shows that the alternative meta-learning approach is not as successful as the proposed CNN. The proposed method makes successful recommendations 78.2% of the time compared to 65.2% with the standard two-step approach (15 of the 23 datasets). From the 15 hits, 11 correspond to the second best model. In this sense, the alternative method is worse than the proposed CNN at identifying the best performing classifier.

4.3 Discussion

The results in Table 1 not only confirm the individual strengths of each classifier for specific types of patterns but also highlight the importance of tailoring classifier selection to the underlying data structure. For instance, SVM consistently performs well for the more complex nonlinear patterns (C3 to C5), likely due to its ability to handle nonlinearity through kernel transformations. In contrast, k -NN shows robust performance in moderately nonlinear settings, benefiting from its local decision boundaries, while RF performs strongly in mixed-noise contexts, leveraging ensemble learning to manage variability. These findings validate our design choice to simulate diverse patterns that mirror real-world classification challenges. Moreover, the results in Table 2 show the capacity of CNN-based meta-learning to internalize and generalize these pattern-specific behaviors across datasets. The near-perfect classification accuracy for CNN1 and CNN2 indicates that the networks effectively captured the latent structure of the tabular representations and learned to associate them with the appropriate classifier profiles. This high performance underscores the benefit of end-to-end learning, which contrasts with the hand-crafted meta-feature approach. In particular, CNN2’s robustness in handling heterogeneous dataset sizes adds practical value, as it reflects the conditions encountered in real-world applications. Finally, the results presented in Tables 5 and 6 highlight the effectiveness of the proposed CNN-based meta-learning approach for classifier recommendation. The CNN model successfully identified one of the top two classifiers in 78.2% of the benchmark datasets, significantly outperforming the expected success rate of a random recommender (33.3%) and surpassing the performance of a traditional two-step meta-learning approach based on decision trees (65.2%). Moreover, the CNN showed high confidence in its pattern predictions, with only three datasets showing a maximum class probability below 0.8. These findings suggest that the CNN model is not only capable of generalizing the patterns

Table 6: Predictions for the tree-based meta-learning approach trained on meta-features.

ID	C1	C2	C3	C4	C5	bc	hit
ds1					✗	k-NN	✗
ds2					✗	SVM	✓(1)
ds3	✗					ANN/SVM	✓(2)
ds4					✗	DT	✗
ds5		✗				k-NN	✓(1)
ds6					✗	RF	✓(2)
ds7					✗	RF	✓(2)
ds8					✗	RF	✓(2)
ds9					✗	k-NN	✗
ds10					✗	k-NN	✗
ds11	✗					RF	✗
ds12		✗				RF	✓(2)
ds13					✗	k-NN	✗
ds14				✗		k-NN	✓(2)
ds15					✗	RF	✓(2)
ds16	✗					logit	✓(1)
ds17					✗	k-NN	✗
ds18					✗	RF	✓(2)
ds19	✗					ANN	✓(2)
ds20		✗				k-NN	✓(1)
ds21		✗				logit/SVM	✗
ds22					✗	RF	✓(2)
ds23					✗	RF	✓(2)

learned from synthetic data to real-world datasets but also more effective than standard meta-feature-based methods in identifying suitable classifiers. There are some methodological challenges that can be improved. For example, more sophisticated strategies can be used to match datasets of different sizes to a CNN, such as spatial pyramid pooling (SPP). This method consists of a special type of pooling layer that allows the inclusion of images of heterogeneous sizes. An SPP layer pools the features in an image and generates outputs that feed the dense layers [28]. Additionally, different CNN architectures could be explored. These challenges, however, would have a minor impact on the proposed framework because the classification accuracy is already near 100% on the simulated datasets.

5 Conclusions

This paper proposes a novel meta-learning approach in which CNNs are used to learn directly from tabular datasets without the need to construct meta-features. The goal is to avoid any loss of information that results from a two-step approach. The proposed model can make accurate classifier recommendations for this type of dataset, outperforming a two-step approach based on meta-features. The main challenge is to define a comprehensive set of simulated patterns that can be extrapolated to real-world datasets. Results show that CNNs used in a supervised manner can achieve nearly perfect identification of simulated patterns; however, the capacity of the proposed model to extrapolate this knowledge to real-world datasets requires further research. Fortunately, the experiments in this study show promising results.

One limitation of the proposed approach lies in the potential risk of overfitting, particularly in the meta-learning context where the goal is to generalize knowledge across different tasks. In the first stage of the framework—training the various meta-learning models—both CNN-based approaches achieved near-perfect classification performance. However, in the second stage, when applying these models to real benchmark datasets, a performance drop was observed due to the presence of noise and lack of clear patterns. Despite this, the results remain positive and indicate that the learned representations from the first stage generalize well. To mitigate overfitting risks, especially the concern of models overfitting to simulated data, noise was introduced into the training datasets by varying sample sizes and adding irrelevant variables, which led to the more robust CNN2 model. Another limitation is the reliance on standard CNNs for meta-feature learning, which inherently have polynomial-time complexity with respect to the input size. Specifically, a typical CNN layer has computational complexity of $\mathcal{O}(n \cdot k^2 \cdot c \cdot m)$, where n is the number of output feature maps, k is the kernel size, c the number of input channels, and m the spatial dimension of the input. Although the simulated tabular inputs in our case are relatively small, this complexity could become a concern in larger-scale applications. Future work

could explore efficient CNN implementations and distributed computing techniques to improve scalability and enhance the practical deployment of meta-learning strategies in more demanding settings. This study represents an initial step toward developing a recommender system that can challenge the NFL theorem. Therefore, this study opens interesting lines for future research. Apart from the identification of new patterns that can be simulated and subsequently linked to classifiers, the proposed method can be used to learn from more sophisticated data structures, such as text or images. The proposed model can also be tailored to the class-imbalance problem, in which classifiers and resampling strategies can be recommended. Finally, generative models can be considered to provide a wider diversity of training patterns.

Acknowledgements

We gratefully acknowledge financial support from ANID, PIA/PUENTE AFB230002 and FONDECYT-Chile, grants 1250045 and 11200007. We are also grateful to the anonymous reviewers who contributed to improving the manuscript.

References

- [1] C. C. Aggarwal. *Neural Networks and Deep Learning*. Springer, 2018.
- [2] E. Alcobaça, F. Siqueira, A. Rivolli, L. P. F. Garcia, J. T. Oliva, and A. C. P. L. F. de Carvalho. Mfe: Towards reproducible meta-feature extraction. *Journal of Machine Learning Research*, 21(111):1–5, 2020.
- [3] S. Ali and K. A. Smith-Miles. On learning algorithm selection for classification. *Applied Soft Computing*, 6:119–138, 2006.
- [4] C. Blake and C. Merz. UCI repository of machine learning databases, department of information and computer science, university of california, irvine, ca, 1998, 2015.
- [5] I.-D. Borlea, R.-E. Precup, and A.-B. Borlea. Improvement of k-means cluster quality by post processing resulted clusters. *Procedia Computer Science*, 199:63–70, 2022.
- [6] L. B. de Amorim, G. D. Cavalcanti, and R. M. Cruz. Meta-scaler: A meta-learning framework for the selection of scaling techniques. *IEEE Transactions on Neural Networks and Learning Systems*, 36(3):4805–4819, 2024.
- [7] I. Deeva and A. Kropacheva. To select or not to select? the role of meta-features selection in meta-learning tasks with tabular data. In *International Conference on Computational Science*, pages 294–308. Springer, 2025.
- [8] N. O. Do Gan and M. Pekdemir. Early detection of mortality in covid-19 patients through laboratory findings with factor analysis and artificial neural networks. *Science and Technology (ROMJIST)*, 25(3-4):290–302, 2022.
- [9] M. Feurer, J. T. Springenberg, and F. Hutter. Using meta-learning to initialize bayesian optimization of hyperparameters. In *MetaSel@ ECAI*, pages 3–10. Citeseer, 2014.
- [10] I. Gabbay, B. Shapira, and L. Rokach. Isolation forests and landmarking-based representations for clustering algorithm recommendation using meta-learning. *Information Sciences*, 574:473–489, 2021.
- [11] T. Hastie, R. Tibshirani, and J. Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.
- [12] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):5149–5169, 2021.
- [13] A. Karatzoglou, N. Schnell, and M. Beigl. Applying depthwise separable and multi-channel convolutional neural networks of varied kernel size on semantic trajectories. *Neural Computing and Applications*, 32(11):6685–6698, 2020.
- [14] I. Khan, X. Zhang, M. Rehman, and R. Ali. A literature survey and empirical study of meta-learning for classifier selection. *IEEE Access*, 8:10262–10281, 2020.
- [15] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2017.
- [16] C. Lemke and B. Gabrys. Meta-learning for time series forecasting and forecast combination. *Neurocomputing*, 73(10):2006–2016, 2010.
- [17] D. Liu, L. Zhang, X. Lai, and H. Liu. Image feature selection embedded distribution differences between classes for convolutional neural network. *Applied Soft Computing*, 131:109715, 2022.
- [18] A. C. Lorena, A. Maciel, P. I. de Miranda, I. G. B. C. Costa, and R. B. C. Prudencio. Data complexity meta-features for regression problems. *Machine Learning*, 107:209–246, 2018.

- [19] E. Lughofer, P. Zorn, and E. Marth. Transfer learning of fuzzy classifiers for optimized joint representation of simulated and measured data in anomaly detection of motor phase currents. *Applied Soft Computing*, 124:109013, 2022.
- [20] D. Michie, D. J. Spiegelhalter, and C. C. Taylor. *Machine learning, neural and statistical classification*. Overseas Press, 2009.
- [21] M. A. Morid, A. Borjali, and G. Del Fiol. A scoping review of transfer learning research on medical image analysis using imagenet. *Computers in Biology and Medicine*, 128:104115, 2021.
- [22] T. Mu, H. Wang, C. Wang, Z. Liang, and X. Shao. Auto-cash: A meta-learning embedding approach for autonomous classification algorithm selection. *Information Sciences*, 591:344–364, 2022.
- [23] J. Ángel González, L.-F. Hurtado, and F. Pla. Twilbert: Pre-trained deep bidirectional transformers for spanish twitter. *Neurocomputing*, 426:58–69, 2021.
- [24] D. Oreski, S. Oreski, and B. Klicek. Effects of dataset characteristics on the performance of feature selection techniques. *Applied Soft Computing*, 52:109–119, 2017.
- [25] A. L. D. Rossi, A. C. P. de Leon Ferreira de Carvalho, C. Soares, and B. F. de Souza. Metastream: A meta-learning based method for periodic algorithm selection in time-changing data. *Neurocomputing*, 127:52–64, 2014.
- [26] P. J. Rousseeuw and M. Hubert. Robust statistics for outlier detection. *Wiley interdisciplinary reviews: Data mining and knowledge discovery*, 1(1):73–79, 2011.
- [27] I. Salehin, M. S. Islam, P. Saha, S. Noman, A. Tunj, M. M. Hasan, and M. A. Baten. Automl: A systematic review on automated machine learning with neural architecture search. *Journal of Information and Intelligence*, 2(1):52–81, 2024.
- [28] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia. Deepfakes and beyond: A survey of face manipulation and fake detection. *Information Fusion*, 64:131–148, 2020.
- [29] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K. Santosh. Advances and challenges in meta-learning: A technical review. *IEEE transactions on pattern analysis and machine intelligence*, 46(7):4763–4779, 2024.
- [30] S. M. Vijithananda, M. L. Jayatilake, B. Hewavithana, T. Gonçalves, L. M. Rato, B. S. Weerakoon, T. D. Kalupahana, A. D. Silva, and K. D. Dissanayake. Feature extraction from mri adc images for brain tumor classification using machine learning techniques. *Biomedical engineering online*, 21(1):52, 2022.
- [31] G. Wang, Q. Song, X. Zhang, and K. Zhang. A generic multilabel learning-based classification algorithm recommendation method. *ACM Trans. Knowl. Discov. Data*, 9(1):1–30, 2014.
- [32] Q. Xia, F. Lee, and Q. Chen. Tcc-net: A two-stage training method with contradictory loss and co-teaching based on meta-learning for learning with noisy labels. *Information Sciences*, 639:119008, 2023.
- [33] F. Xing, Y. Xie, H. Su, F. Liu, and L. Yang. Deep learning in microscopy image analysis: A survey. *IEEE transactions on neural networks and learning systems*, 29(10):4550–4568, 2017.
- [34] Y. Yamada, O. Lindenbaum, S. Negahban, and Y. Kluger. Feature selection using stochastic gates. In H. D. III and A. Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10648–10659, 13–18 Jul 2020.
- [35] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu. Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11):3212–3232, 2019.
- [36] X. Zhu, X. Yang, C. Ying, and G. Wang. A new classification algorithm recommendation method based on link prediction. *Knowledge-Based Systems*, 159:171–185, 2018.